

FlaskGuard

Custom Web Application Firewall

Technical Report – Part 1

Flask Foundation, frontend

Repository:	<code>github.com/echoenvoy/FlaskGuard</code>
Report Part:	1 of 5
Author:	Saoud Amine
Primary Language:	Python 3, HTML5, JavaScript
Framework:	Flask (Pallets)
Date:	June 2026

Contents

1	Introduction and Project Overview	2
1.1	What is FlaskGuard?	2
1.2	Five-Part Structure	2
1.3	Scope of This Report	2
2	Flask Application Scaffold	3
2.1	Framework Selection Rationale	3
2.2	Application Entry Point – <code>app.py</code>	3
2.3	Inspection Pipeline Order	3
2.4	Project Directory Structure	4
3	Dependencies and Requirements	4
3.1	<code>requirements.txt</code> Analysis	4
4	Conclusion	4

Introduction and Project Overview

What is FlaskGuard?

FlaskGuard is a lightweight, educational Web Application Firewall (WAF) and reverse proxy implemented entirely in Python using the Flask microframework. It is designed to intercept all incoming HTTP traffic directed at a protected backend service, inspect each request for malicious patterns, and either forward the clean request to the backend or block it with an appropriate HTTP error response.

The project demonstrates, in a self-contained and well-documented codebase, the core architectural ideas that underpin commercial WAF products such as Cloudflare, AWS WAF, and ModSecurity. Unlike those enterprise solutions, FlaskGuard is intended for learning, experimentation, controlled internal deployments, and as a portfolio demonstration of applied security engineering.

The complete traffic flow through the system is as follows:

Visitor → **FlaskGuard** → Protected Backend

Five-Part Structure

The project is divided into five development phases:

Part	Scope
1	Flask application scaffold, YAML configuration, rate limiting with <code>flask-limiter</code> , and a custom HTML/JS/CSS Admin Security Dashboard built from scratch.
2	Regex-based attack detection engine, structured attack logger, and Docker containerisation.
3	Machine learning classifier (Naive Bayes with character n-gram TF-IDF), IP management subsystem (strikes, auto-bans, manual ban/unban).
4	File upload scanner (magic bytes, extensions, archives, Office files, SVGs), HTTPS/TLS encryption, and secrets hardening.
5	Full reverse proxy implementation, demo backend application, and a comprehensive test suite.

Scope of This Report

This document addresses **Part 1** in full technical depth. It covers:

- The Flask application factory and application entry point (`app.py`)
- The YAML configuration schema (`config.yml`) and environment variable overrides
- Rate limiting integration via `flask-limiter`
- The Admin Security Dashboard: both the backend API routes and the custom frontend
- HTTP Basic Authentication protecting the `/admin` namespace
- Design decisions, trade-offs, and recommendations

Flask Application Scaffold

Framework Selection Rationale

Flask was selected as the implementation framework for several well-considered reasons:

- **Minimal surface area:** Flask provides only what is necessary (WSGI adapter, routing, Jinja2 templating, and request/response primitives), keeping the WAF dependency footprint small.
- **Middleware-friendly design:** Flask's request lifecycle (before-request hooks, after-request hooks, error handlers) maps naturally to the WAF inspection pipeline.
- **WSGI compatibility:** Any WSGI-compatible server (Gunicorn, uWSGI, Waitress) can serve FlaskGuard without code changes.
- **Ecosystem maturity:** Libraries such as `flask-limiter`, `requests` (for the proxy), and `PyYAML` integrate cleanly with Flask without framework-level conflicts.

Application Entry Point – `app.py`

The main application file (`flaskguard/app.py`) serves multiple roles:

1. **Flask instance creation:** Instantiates the Flask application object, configuring template and static file paths.
2. **Configuration loading:** Calls the configuration subsystem to parse `config.yml` and merge environment variable overrides.
3. **Extension initialization:** Sets up `flask-limiter` with the loaded rate-limiting configuration.
4. **Before-request pipeline:** Registers `@app.before_request` hooks to implement the WAF inspection pipeline in order: admin auth check, IP blacklist check, file upload scanning, regex detection, and ML classifier fallback.
5. **Route definitions:** Defines `/admin` and all `/admin/api/*` routes for the dashboard, plus the catch-all `/<path:path>` proxy route.
6. **Entry point guard:** A `if __name__ == "__main__"` block starts the development server with the configured port and debug flag.

Inspection Pipeline Order

A critical design decision is the **order** in which security checks are applied. FlaskGuard processes each request through these layers in sequence:

Step	Action
1	Admin Basic Auth check (for <code>/admin</code> namespace)
2	IP blacklist check (immediate 403 for banned IPs)
3	File upload scanner (if <code>Content-Type</code> indicates multipart data)
4	Regex-based attack pattern detection
5	ML classifier (fallback for inputs that pass the regex layer)
6	Forward clean requests to backend via reverse proxy

The order is deliberate: cheap checks (auth, blacklist) run before expensive ones (ML inference), minimising computation for already-blocked IPs. File scanning runs before regex because a malicious filename may not trigger string-based patterns.

Project Directory Structure

The package layout follows Flask application conventions:

```

1 FlaskGuard/
2 |-- flaskguard/
3 |   |-- app.py           # Main Flask app (entry point, routes,
   |   | pipeline)
4 |   |-- security.py      # Regex attack detection
5 |   |-- ml_classifier.py  # Naive Bayes ML model
6 |   |-- file_scanner.py  # Upload scanner
7 |   |-- ip_manager.py    # IP blacklist logic
8 |   |-- attack_logger.py # JSON + file logging
9 |   |-- proxy.py         # Reverse proxy forwarding
10 |   '-- templates/
11 |       '-- dashboard.html # Admin dashboard (all-in-one HTML/CSS/JS)
12 |-- demo_backend/        # Example backend protected by FlaskGuard
13 |-- logs/                # Sample log files
14 |-- models/              # Persisted ML model files
15 |-- config.yml            # Main configuration
16 |-- docker-compose.yml   # WAF + demo backend composition
17 |-- Dockerfile           # Container image definition
18 '-- requirements.txt      # Python dependencies

```

Listing 1: FlaskGuard directory tree

Dependencies and Requirements

requirements.txt Analysis

The following is an analysis of the core Part 1 dependencies declared in `requirements.txt`:

Package	Role
flask	Core web framework: routing, request handling, Jinja2 templating
flask-limiter	Per-IP rate limiting with pluggable storage backends
pyyaml	Parsing <code>config.yml</code> into Python dictionaries
requests	HTTP client used by the reverse proxy to forward requests
scikit-learn	ML classifier (Naive Bayes + TF-IDF) – Part 3
joblib	Model serialisation (<code>.pkl</code> files) – Part 3
python-magic	Magic byte detection for file uploads – Part 4

Conclusion

Part 1 of the FlaskGuard project establishes a solid, well-structured foundation for the WAF system. The choice of Flask as the host framework provides a minimal, WSGI-

compatible base that integrates cleanly with the rate-limiting, authentication, and dashboard components. The YAML configuration system gives operators comprehensive control over runtime behaviour without requiring code changes, while environment variable overrides support secure container-based deployments following the Twelve-Factor App pattern.

The Admin Security Dashboard demonstrates how a functional, real-time security monitoring interface can be built with vanilla HTML, CSS, and JavaScript, served directly from the Flask application, without any frontend build toolchain. The clean separation between backend API endpoints (`/admin/api/*`) and the dashboard UI (`/admin`) makes the system extensible and testable.

The rate limiting integration provides a first line of defence against brute-force and denial-of-service attacks, and the HTTP Basic Auth gate prevents unauthorised access to the sensitive admin interface. Together, these components form a coherent, deployable foundation that the remaining four parts build upon.